QualiAdept

Session 5 - Async JavaScript

Chapter 1: Synchronous vs. Asynchronous and the Event Loop Architecture 2
Learning Objectives 3
1.1 The Nature of JavaScript: Single-Threaded (Synchronous Execution) 3
1.2 The Savior: Asynchronous Execution (Non-Blocking) 4
1.3 Hidden Superpowers: Web APIs 7
1.4 Internal Architecture: How does the Event Loop work? 8
1.5 🤖 Engineer Mindset in the AI Era: The Connection with Playwright 10
Chapter 2: Managing Asynchrony - From "Callback Hell" to Promises and async/await 10
Learning Objectives 11
2.1 The Dark Age of JavaScript: "Callback Hell" 11
2.2 The Rescue: What is a Promise and who invented it? 14
2.3 The 3 Strict States of a Promise 14
2.4 The 2017 Revolution: The async and await syntax 16
2.5 Did you know...? 17
2.6 🧠 Technical Interview Question / Challenge 17
2.7 🤖 Engineer Mindset in the AI Era: "Unhandled Promise Rejection" 18
Chapter 3: Client-Server Communication and the Fetch API (Fetching data over the network) 19
Learning Objectives 20
3.1 Client-Server Architecture and the "Waiter" (The API) 20
3.2 The fetch() method and the "Pause" button (await) 21
3.3 Extracting data: Why do we need a second await? 22
3.4 The Safety Net: The mechanics of the try...catch block 23
3.5 Did you know...? 24
3.6 🧠 Interview Challenge / Practical Execution 25
3.7 🤖 Engineer Mindset in the AI Era: "Don't test blind, test completely" 25
Chapter 4: Advanced Concepts (Bonus) - Beyond GET: Sending data (The POST Method) 26

Learning Objectives....26
4.1 When "reading" is not enough (GET vs. POST)....27
4.2 Configuring fetch to send data....27
4.3 Practical Example: Creating a new Task on the Server....28
4.4 🤖 Engineer Mindset in the AI Era: API Setup....29
Chapter 5: Session 5 Assignment & Extra Practice (Towards Senior Level)....30
5.1 Homework Assignment (Session 5 Project)....30
5.2 Bonus Challenge (Optional): "Loading..." Effect and Safety (try/catch)....31
5.3 🔑 Solution Keys (Answers & Explanations)....32
Live Coding Manual: Session 5 - Asynchronous JavaScript & Fetch API....34
Part 1: The Playground (UI Sandbox)....34
Part 2: Live Exercises....36
🔴 For Chapter 1: The Blocking Demonstration (Sync vs Async)....36
🔴 For Chapter 2: Promises & async/await....37
🔴 For Chapter 3: The First Network Request (Fetch GET)....38
🔴 For Chapter 4 (Bonus): Sending Data to the Server (Fetch POST)....39
🔴 For Chapter 5: End-to-End & Controlled Failure (try/catch)....40
Advanced Workbook: Session 5 - Asynchronous JavaScript (Extra Practice)....42
Part 1: Application Setup (The Sandbox)....42
Part 2: The Exercise Suite (Run in F12 Console)....44
💻 Exercise 1: The Sync Nightmare (Event Loop Blocked)....44
💻 Exercise 2: Asynchronous Chaining (setTimeout)....45
💻 Exercise 3: Creating a Promise from Scratch....46
💻 Exercise 4: Consuming the Promise (async/await)....46
💻 Exercise 5: Fetch GET - Bringing Real Data....47
💻 Exercise 6: Fetch POST - Reporting a Bug over the Network....48
💻 Exercise 7: Resilience with Try/Catch....49
💻 Exercise 8: Advanced Concept - Parallelization (Promise.all)....50

Chapter 1: Synchronous vs. Asynchronous and the Event Loop

Architecture

Learning Objectives

🎯

- Understand the fundamental difference between Synchronous (blocking) and Asynchronous (non-blocking) execution through real-life examples.
- Discover what Web APIs are, why they appeared, and how they give JavaScript "superpowers".
- Visualize the browser's internal mechanism: Call Stack, Callback Queue, and Event Loop.
- Correlate asynchronous theory with the reason Playwright tests constantly need the await keyword.

1.1 The Nature of JavaScript: Single-Threaded (Synchronous Execution)

"Out of the box", JavaScript is a **Single-Threaded** language. This means it has a single main processor that can execute **only one line of code at a time**. The code is read from top to bottom, sequentially. This is **Synchronous (Blocking) Execution**.

Example 1: The Synchronous Coffee Shop

📝 Imagine a coffee shop with a single barista. They take your order (a complicated Frappuccino that takes 5 minutes). Instead of asking you to wait at a table and taking the next customer's order, the barista blocks: they stand and prepare your drink, while a huge line forms at the door. No one else is served.



> 📈 In the web world, if a line of code takes 10 seconds to download a huge image synchronously, the entire web page "freezes" for 10 seconds. You can't click, you can't scroll. It is a catastrophic scenario for user experience.

1.2 The Savior: Asynchronous Execution (Non-Blocking)

To avoid blocking the page, JavaScript uses **Asynchronism**. This allows initiating a slow action, delegating it to someone else, and immediately continuing the execution of the rest of the code.

Example 2: Ordering Pizza (Real Life)

🤔 It's Friday night, you want to clean the house, but you are also hungry. You have a choice in how to proceed:

- **Synchronous Approach (Wrong):** You call the pizzeria, order the pizza. Then, you go to the door and stand there, motionless, waiting 40 minutes for the delivery guy. When he arrives, you eat, and only then do you start vacuuming. You wasted 40 minutes for nothing!
- **Asynchronous Approach (Smart - How JS does it):** You call the pizzeria and order (initiate the action). You provide your phone number and address (set a *Callback*). You hang up and immediately start vacuuming (the code continues to run, you are not blocked). When the pizza is ready, the doorbell rings (an *Event* occurs). You pause the vacuum, take the pizza, eat it (execute the *Callback*), and then return to cleaning.

QualiAdept

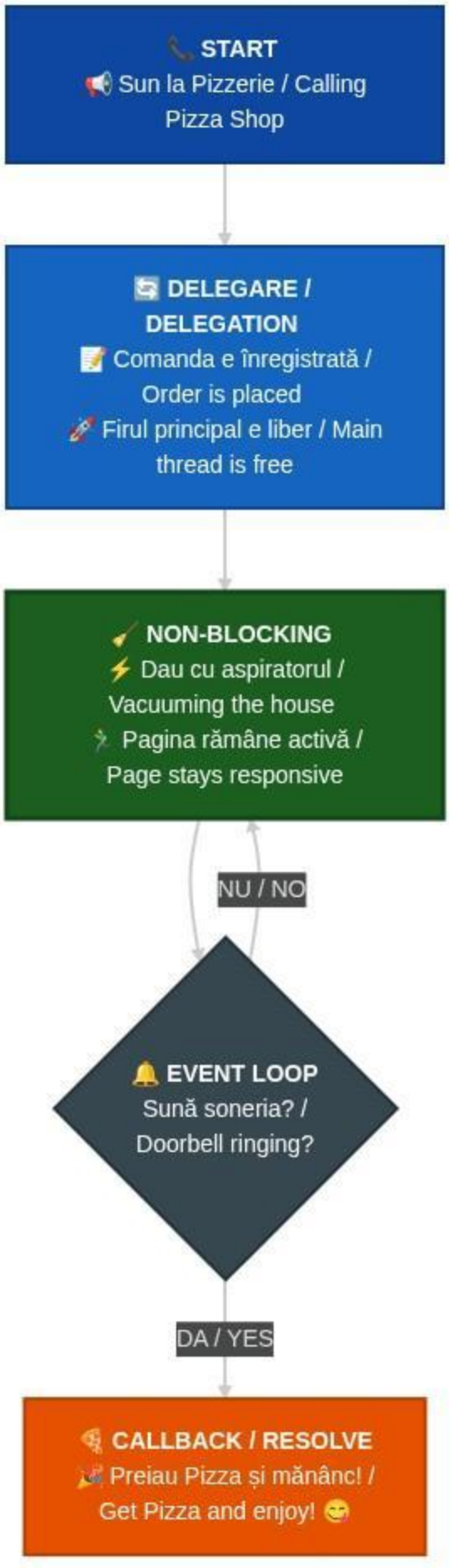

📞 START
📢 Sun la Pizzerie / Calling Pizza Shop
🔄 DELEGARE / DELEGATION
📝 Comanda e înregistrată / Order is placed
🚀 Firul principal e liber / Main thread is free
🧹 NON-BLOCKING
⚡ Dau cu aspiratorul / Vacuuming the house
🏃 Pagina rămâne activă / Page stays responsive
NU / NO
🔔 EVENT LOOP
Sună soneria? / Doorbell ringing?
DA / YES
🍕 CALLBACK / RESOLVE
🎉 Preiau Pizza și mănânc! / Get Pizza and enjoy! 😋

Fig. 1.2 - The diagram above illustrates the inner workings of an **asynchronous**, event-driven model, which is the exact opposite of the blocking coffee shop scenario:

- **The Problem Solved:** You no longer waste time on the phone or by the door waiting for the pizza to bake. The heavy task (preparation and delivery) has been delegated to the background.
- **Web Analogy:** In web development, this is the core of asynchronous programming. When you trigger a time-consuming action (like downloading an image or fetching data from an API), the browser doesn't freeze. The main thread delegates the task to the background and remains completely free to respond to user interactions (scrolling, clicking, animations) while "vacuuming". Once the asset is ready, an event is fired (the doorbell rings), triggering the callback that displays the result on screen.

1.3 Hidden Superpowers: Web APIs

If JavaScript has a single thread of execution, how can it do multiple things at the same time (like counting 3 seconds with setTimeout and simultaneously listening for a button click)?

The secret is that **JavaScript DOES NOT do them!** JS itself is a limited language, which only knows math, variables, and functions. It doesn't know how to communicate with the internet, it doesn't know how to manipulate the DOM, and it has no internal clock.

> 🧠 Who does the heavy lifting? **The environment where JS runs (the Browser or Node.js).**

The browser (Chrome, Firefox) is a complex program written in C++, which has hundreds of "threads". The browser grants JavaScript access to a set of tools called **Web APIs** (Application Programming Interfaces).

- **When did they appear?** In the 2000s, Microsoft introduced the first API to make background network requests (XMLHttpRequest). This led to the explosion of the **AJAX** era (Web 2.0). Google Maps and Gmail were among the first to use these asynchronous APIs: you could drag the map and new images would download instantly, without the page refreshing itself!
- **Examples of Web APIs:** setTimeout (Browser timer), document.querySelector (DOM API),

fetch (Network/Internet), Geolocation, LocalStorage.

1.4 Internal Architecture: How does the Event Loop work?

The system that orchestrates the perfect communication between your JS code and the browser's Web APIs consists of 4 major components. Follow the diagram carefully:



Fig. 1.3 - The diagram above displays the exact architecture that allows JavaScript to handle heavy operations without freezing the website's user interface:

- **Call Stack:** The place where JS runs the current code, synchronously. It works on the LIFO (Last In, First Out) principle.

- **Web APIs:** When your code encounters an asynchronous action, JS "passes" it to the browser. The browser starts its own timer or network request and instantly frees up the Call Stack, allowing JS to continue running.
- **Callback Queue:** When the browser finishes downloading data from the network, it DOES NOT interrupt the currently running JS code. It puts the response and your function in a "queue" (on the FIFO - First In, First Out principle).
- **Event Loop:** It is a continuous process (an infinite *while loop*) that does one single job: It looks at the Call Stack and the Callback Queue. **The Golden Rule:** The Event Loop will move a task from the Queue back into the Call Stack to be processed ONLY the very second the Call Stack is completely EMPTY.

1.5 🤖 Engineer Mindset in the AI Era: The Connection with Playwright

> 🌓 **QA vs. AI: Why do tests fail without asynchronism?**
>
> - **What actually happens:** Beginners (and sometimes incorrectly generated AI code) write UI commands tested "blindly":
>   1. page.click('#login-button')
>   2. expect(errorMessage).toBeVisible()
> - **The Problem:** An AI without framework context will treat the code above Synchronously. But interacting with a real browser over the network is Asynchronous (Web API). When JS executes line 1, it delegates it to the browser and moves *instantly*, in zero milliseconds, to line 2. Obviously, the error message hasn't appeared on the screen yet (because the request has barely left for the server), and the test falsely fails.
> - **How a QA engineer thinks:** The engineer knows the Event Loop. They know they have to keep JS "in check". Using modern Asynchronous syntax (async / await), they tell the test code: *"Wait (await)! Send the click command to the Web API, but pause my testing Call Stack until the Event Loop reports that the browser has finished processing the action. Only then proceed to line 2."*

Chapter 2: Managing Asynchrony -

From "Callback Hell" to Promises and async/await

Learning Objectives

🎯

- Understand the limitations of the old asynchronous system and recognize "Callback Hell".
- Discover what a "Promise" really is, who standardized it, and why.
- Learn the 3 strict states of a Promise and the reasons why there cannot be more.
- Make the transition from clunky .then() chains to the modern and clean async / await syntax.
- Successfully answer classic technical interview scenarios regarding asynchrony.

2.1 The Dark Age of JavaScript: "Callback Hell"

In the previous chapter, we saw that asynchronous functions (like a network request to fetch data) do not return the result instantly. To know when the action finished, for the first 15 years of JavaScript's existence, the only solution was to pass a function (a *Callback*) as a parameter.

📈 *"When you finish downloading the file, please run this callback function."*
It sounded simple until actions became dependent on one another. Let's assume we have an E2E scenario: (1) Authenticate user -> (2) Find their cart ID -> (3) Add a product to the cart. These are all slow network actions. Here is what the code looked like:

```
// Classic example of Callback Hell (The Pyramid of Doom)
loginUser('testUser', 'password123', function(error, user) {
    if (error) {
        console.error("Login error");
    } else {
```

```
        findCart(user.id, function(cartError, cart) {
            if (cartError) {
                console.error("Cart not found");
            } else {
                addProduct(cart.id, 'Phone', function(productError, result)
{
                    if (productError) {
                        console.error("Add to cart error");
                    } else {
                        console.log("Product added successfully!");
                        // ... and the code kept drifting further to the
right
                    }
                });
            }
        });
    }
});
```

The Problem: This code takes the shape of a horizontal pyramid (>). It is dubbed **"Callback Hell"** or the *"Pyramid of Doom"*. It is impossible to read, extremely hard to maintain, and error handling (that repetitive if (error)) had to be done manually at every single step.

Here is how this architectural burden looks visually (notice how each successful step "sinks" you deeper, while you have to separately handle every possible error):

QualiAdept

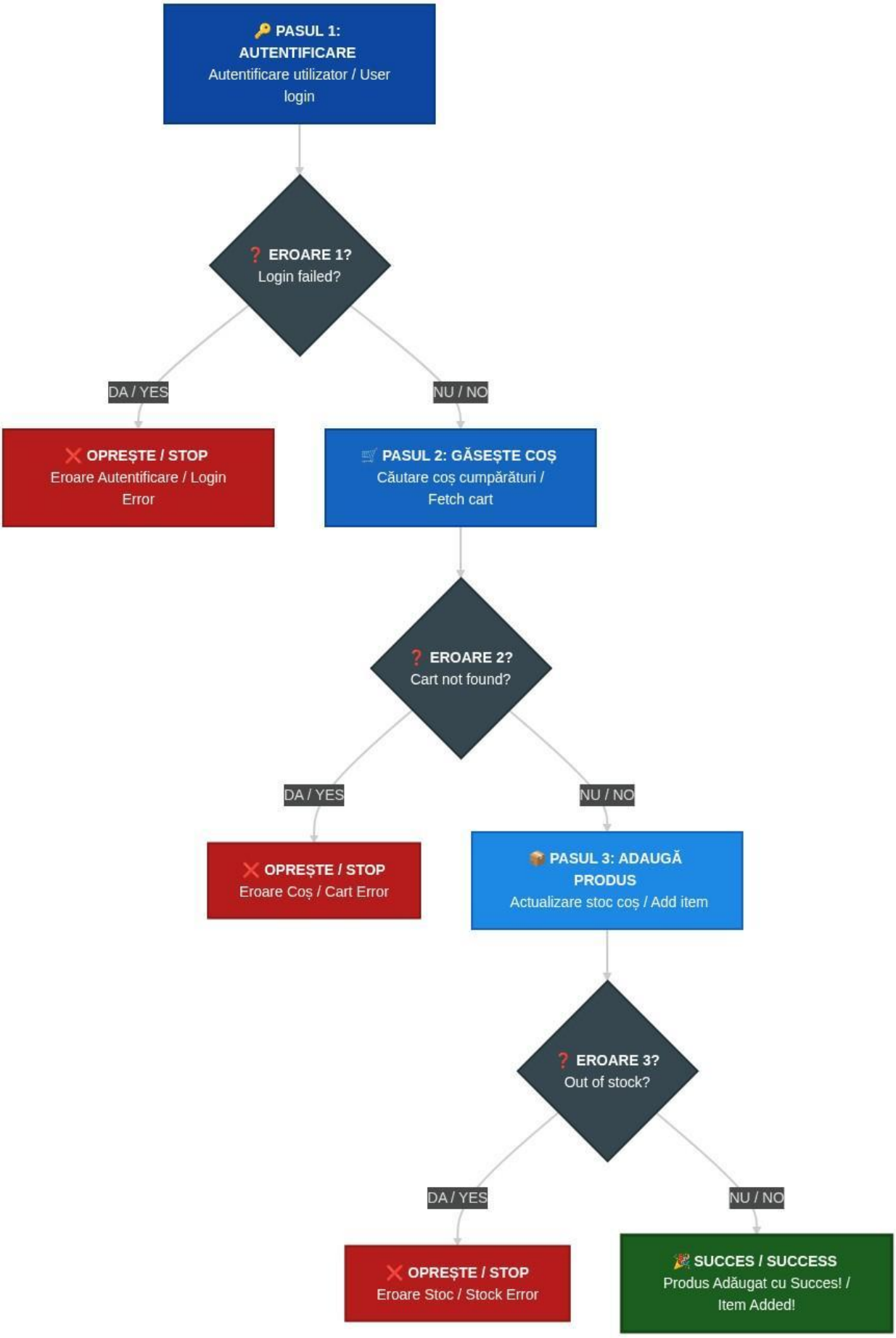

🔑 PASUL 1: AUTENTIFICARE
Autentificare utilizator / User login
❓ EROARE 1?
Login failed?
DA / YES
NU / NO
❌ OPREȘTE / STOP
Eroare Autentificare / Login Error
🛒 PASUL 2: GĂSEȘTE COȘ
Căutare coș cumpărături / Fetch cart
❓ EROARE 2?
Cart not found?
DA / YES
NU / NO
❌ OPREȘTE / STOP
Eroare Coș / Cart Error
📦 PASUL 3: ADAUGĂ PRODUS
Actualizare stoc coș / Add item
❓ EROARE 3?
Out of stock?
DA / YES
NU / NO
❌ OPREȘTE / STOP
Eroare Stoc / Stock Error
🎉 SUCCES / SUCCESS
Produs Adăugat cu Succes! / Item Added!

Fig. 2.1 - The diagram above perfectly captures the architectural nightmare known as **Callback Hell** or the **Pyramid of Doom**:

- **The Visual Issue:** Every successful asynchronous operation forces you to nest another inner callback function. The actual code stretches heavily to the right, making it incredibly exhausting to read or trace.
- **The Maintenance Issue:** Error handling becomes a manual chore. At each individual step, you are strictly required to write a repetitive `if (error)` statement. If you miss handling a single error at step 2, the entire application can crash silently, leaving the user staring at a broken screen.

2.2 The Rescue: What is a Promise and who invented it?

> 💰 To solve this chaos, the open-source community created a standard called the **"Promises/A+ specification"**. Seeing how well it worked, the **TC39** committee (the group of architects deciding the future of JavaScript) adopted the idea and officially introduced the **Promise** object in the ES6 version (in 2015).

Real-Life Example: The online order and the Tracking Number (AWB)

A Promise is a special object in JavaScript that acts as a "waiting ticket" (a proxy) for a value that is unknown at the moment of its creation.

***Analogy**:* When you order an expensive laptop online, you don't receive the laptop instantly through the screen. Instead, you receive a tracking number (AWB) and an email saying: *"We promise to deliver your product."* You can then go on with your life. That tracking number is the equivalent of a *Promise*. The tracking number promises that at some point it will give you a result: either the courier arrives with the laptop at your door, or you are notified that the package was lost.

2.3 The 3 Strict States of a Promise

The TC39 standard decided a critical thing: to guarantee the predictability of systems, a Promise can only go through **3 states**, and once it has finished, its state becomes *immutable* (it can never change again).

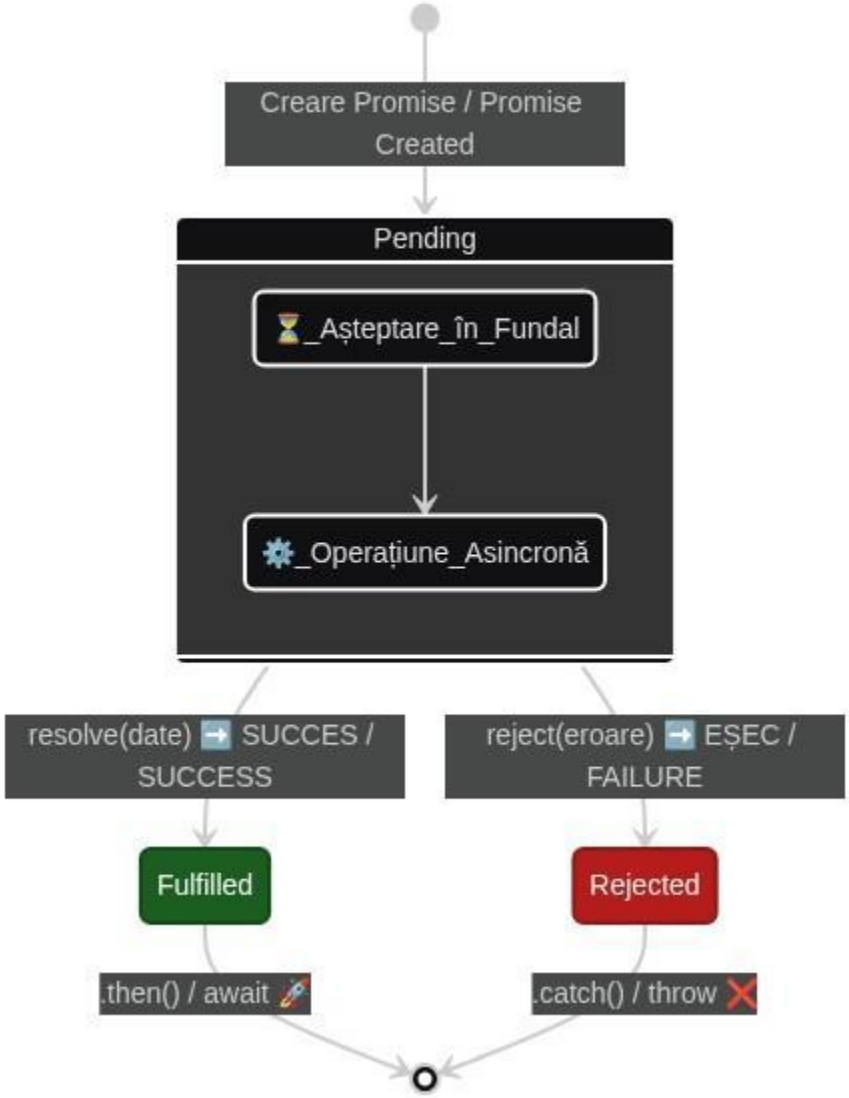


Fig. 2.2 - The 3 Sacred States of a Promise (TC39)

To guarantee the absolute predictability of web software, the TC39 standard established an iron-clad rule: a Promise's state is **immutable** (once resolved or rejected, it can never change again).

- **Pending:** The initial state. You placed the order, but the courier is still on the way. The network request has left for the server, but the result is unknown.
- **Fulfilled / Resolved:** The action finished successfully. The courier arrived with the package.

The server responded (e.g., 200 OK code).

- **Rejected:** The action failed. The courier lost the package or had an accident. The server is down (e.g., 500 Error) or the internet disconnected.

Why isn't there a fourth state (e.g., "Canceled")?

> The TC39 architects wanted Promises to be safe and deterministic. A promise represents the *final* result of an action. If you could externally "cancel" a promise, your code could get stuck in an uncertain state, and the functions waiting for the result wouldn't know what to do. Therefore, it either resolves or rejects. Period.

2.4 The 2017 Revolution: The async and await syntax

Initially, to read a promise, developers used the .then() command. Although it saved the world from "Callback Hell", it created long chains of promises.

> So, in 2017 (the ES8 standard), JS introduced a revolutionary syntax that changed how we write code: **async / await**. This is just "syntactic sugar" (a more aesthetic way of writing) on top of promises, which allows us to write asynchronous code that looks exactly like synchronous code!

- **async:** Placed in front of a function, it guarantees that the function will automatically return a Promise. It also grants permission to use the await command inside it.
- **await:** "Pauses" only the current function. It tells JavaScript: *"Stop execution on this line. Give control back to the browser to do something else, and return here ONLY when the Promise on your right goes from the Pending state to Fulfilled."*

The same scenario from above (Look how clean it is now!):

```
// The logic is flat, easy to read from top to bottom!
const executeFullTest = async () => {
    try {
```

```
        const user = await loginUser('testUser', 'password123');
        const cart = await findCart(user.id);
        const result = await addProduct(cart.id, 'Phone');

        console.log("Product added successfully!");
    } catch (error) {
        // A single Catch block catches any error from any of the 3 steps!
        console.error("Test failed at one of the steps:", error);
    }
};
```

2.5 Did you know...?

💡

- **Promise.all() for parallel execution:** If you have 3 independent network requests (e.g., you want to fetch the user profile, the categories list, and the theme settings), you don't have to wait for them sequentially (which would take, say, 1s + 1s + 1s = 3 seconds). You can use await Promise.all([request1, request2, request3]) to trigger them all simultaneously, reducing the total time to just 1 second! This is a vital technique for speeding up tests.
- **Playwright and Promises:** Almost 99% of the commands you will write in Playwright (e.g., page.click(), page.fill(), page.goto()) return a Promise, because they rely on the network. This is why you will use the await keyword dozens of times in a single test file.

2.6 🤔 Technical Interview Question / Challenge

This is an extremely common question in QA Automation interviews (Mid/Senior level) that tests the understanding of Promises.

🗣️**The Interviewer:** "You are in a Playwright test. You want to extract the text from a button and display it in the console. You write the following code:"

// Note: page.locator().textContent() is an asynchronous Playwright method
const buttonText = page.locator('.btn-login').textContent();

console.log(buttonText);

❌ **The Question:** "When you run the test, instead of displaying 'Log In', the console prints Promise { <pending> }. Why does this happen and how do you fix the code?"

📣 **Your Engineer Answer:**

"It prints Promise { <pending> } because interacting with the browser through Playwright happens over the network (WebSocket). So, the textContent() method doesn't return the text instantly; it returns a *Promise* (a tracking number) that it will fetch the text.

Because the code forgot to use the await keyword, JavaScript did not pause execution. It took the created Promise (which in millisecond 1 is obviously in the *Pending* state), saved it in the variable, and printed it directly to the screen.

☑ **To fix it**, we must add await in front of the command:

const buttonText = await page.locator('.btn-login').textContent(); - only now will the script wait for the promise to become *Fulfilled* and will put the actual text ('Log In') into the variable."

2.7 🤖 Engineer Mindset in the AI Era: "Unhandled Promise Rejection"

> 🌓 **QA vs. AI: Abandoned Promises**
>
> - **What AI does well:** Code generators (Copilot, ChatGPT) know the general rules. If you tell it to write a Playwright test, in the vast majority of cases, it will correctly place async and await.
> - **The Trap (Where AI fails):** Sometimes, when you ask the AI to write a *custom* utility function (for example, a for loop that cleans the database before a test), the AI might write the logic but could "forget" to mark the sub-commands with await. In modern JavaScript, if a Promise fails (state: *Rejected*) and was not properly "awaited" to catch the error, the system will suddenly crash with the famous **Unhandled Promise Rejection** error (A promise was rejected and no one was there to handle the error).

This can cause immense instability in the CI/CD pipeline.

- **How a QA engineer thinks:** When the engineer does a Code Review on a script generated by AI or a Junior colleague, their eye exclusively scans for a specific rule: *Do we have any Web API method called without an await in front of it?* The engineer enforces flawless asynchronous architecture to ensure a deterministic and safe flow.

Chapter 3: Client-Server Communication and the Fetch API (Fetching data over the network)

Learning Objectives

🎯

- Understand what an API (Application Programming Interface) is and how the Client-Server architecture works.
- Use the native fetch() method to make real HTTP requests to a server and understand its parameters.
- Combine fetch() with async / await to "pause" the code, download, and read data in JSON format.
- Discover the internal mechanics of try / catch blocks for handling fatal network errors.
- Correlate these concepts with Playwright API testing.

3.1 Client-Server Architecture and the "Waiter" (The API)

Up to this point, our application has only lived in the browser (Frontend). But in a real application, the list of products, users, and passwords are not stored in the HTML file, but on a powerful remote computer called a **Server** (or Backend/Database).

When you (the Client) want to see the product list, your browser has to send a message over the internet to the Server, and the Server has to send the data back.

What is an API?

API (Application Programming Interface) is the bridge between the Frontend and the Backend.

QA Analogy: You are at a restaurant. You are the **Client** (Frontend/Browser). The kitchen is the **Server** (Backend where data is prepared). You are not allowed to go directly into the kitchen to get your food. You need a **Waiter**. That waiter is the **API**. They take your order (Request), carry it to the kitchen, and bring your food back (Response).

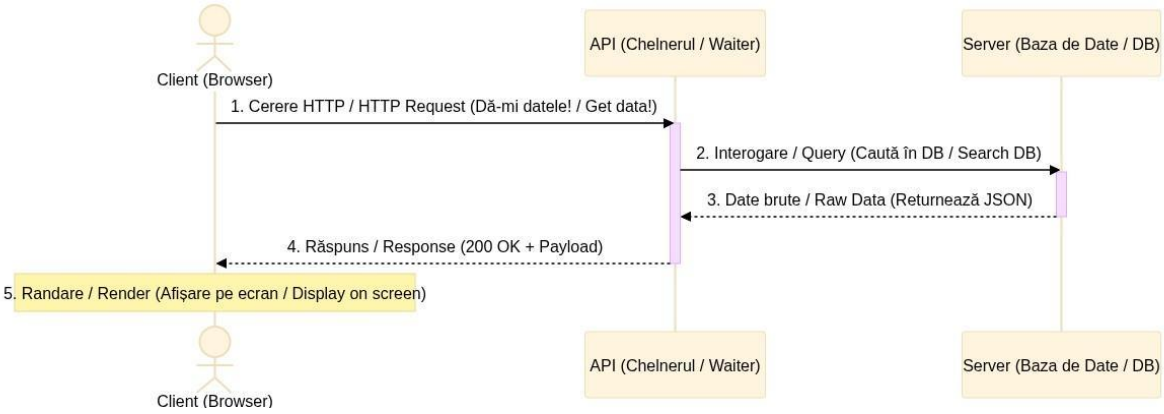


Fig. 3.1 - The sequence diagram above illustrates the chronological steps data goes through from the exact millisecond a user interacts with a page until the content lights up the screen:

- **The Initial Request (Steps 1-2):** The browser (Client) fires a digital letter to the API (the Waiter). The API doesn't store the products locally, so it steps into the backroom to query the source of truth (the Server/Database).
- **Asynchronous Processing (Steps 3-5):** The database transfers the raw structural payload back to the API (formatted as JSON text). The API elegantly wraps this response with a success stamp (200 OK) and hands it over to the browser. Throughout this entire roundtrip, if written asynchronously, the user interface remained completely fluid. Finally, the browser unpacks the JSON data and renders (paints) the visual elements onto the viewport.

3.2 The fetch() method and the "Pause" button (await)

In modern JavaScript, the native tool (without installing external libraries) used to send an HTTP request over the network is called **fetch()**.

How does fetch() work?

> 🧠 In its simplest form, fetch() needs only one mandatory parameter: the **URL (address)** from where you want to fetch the data. By default, if you don't tell it otherwise, fetch() will make a **GET** request (meaning it only asks for data, it doesn't delete or modify anything on the server).
> Because sending a message over the internet to a server in the US and back is a slow action, the fetch() method always returns a **Promise** (an "AWB" tracking number telling you the request

has left).

The power of await:

Here is where the magic of await comes in. Without it, JavaScript would send the request and instantly move to the next line, before the data even arrived! By putting await in front of fetch(), you press a "Pause" button on the remote control. You tell the code: *"Stop on this line! Do not take another step until the Promise returns from the internet with a Fulfilled (successful request) or Rejected (server down) state."*

```
// An asynchronous function that fetches data from a web address
const fetchServerData = async () => {
    console.log("1. The request left for the server...");

    // The function execution STOPS here until the response comes back
    const response = await
fetch('https://jsonplaceholder.typicode.com/users/1');

    // This line executes ONLY AFTER the response has arrived
    console.log("2. The raw response has arrived!", response);
};
```

3.3 Extracting data: Why do we need a second await?

If you run the code above, you will see that the response variable does not contain the username you are looking for. It contains a huge HTTP object, showing you the Status (e.g., 200) and some network information.

To read the actual data inside this package, we need to unpack it. Data almost always comes in **JSON** format (JavaScript Object Notation - a text structure that looks exactly like JS objects).

To transform the text coming from the network into a JS object that we can read, we use the .json() method.

Attention! Because the data package (payload) coming from the server can be 2 Megabytes and might take some time to be unpacked/translated, the .json() command is also an asynchronous action. So it **returns a second Promise**, which is why it needs its own await!

This is how to write a complete HTTP request correctly in 2026:

```
const displayUser = async () => {
    // Step 1: Wait for the network journey to finish
    const httpResponse = await
fetch('https://jsonplaceholder.typicode.com/users/1');

    // Step 2: Wait for the data to be unpacked from HTTP into a JS object
(JSON)
    const user = await httpResponse.json();

    // Step 3: Now we have a real Object and we can access its properties!
    console.log("The user's name is:", user.name);
    console.log("His email is:", user.email);
};
```

3.4 The Safety Net: The mechanics of the try...catch block

On the internet, things frequently go wrong. The server goes down, the user has no signal on their phone, or the link (URL) is misspelled. If a Promise fails (state: *Rejected*) and we don't catch the error, the application completely freezes (fatal crash).

To prevent this, programmers use an emergency mechanism called **try...catch**.

How it works:

- **try { ... }:** Here you put all your main code, the "Happy Path". JS tries to execute the code line by line. If everything goes well, the catch block is completely ignored.
- **The invisible crash:** If *absolutely any* command inside the try block fails (e.g., the internet drops right when fetch() happens), the execution of the code inside try **stops instantly** on that line. The following lines inside try are not executed!
- **catch (error) { ... }:** The script immediately "jumps" into this block. The error variable will contain the detailed technical message about what went wrong. Here you, as an engineer,

decide how to handle the situation: display a friendly message to the user ("We are sorry, please try again") or fail the test in a controlled manner.

```
const requestSafeData = async () => {
    try {
        console.log("A. Starting transaction..."); // Executes

        // If the URL is wrong or the network drops, this line THROWS AN
ERROR!
        const response = await
fetch('https://jsonplaceholder.typicode.com/users/9999');

        // GOOD QA PRACTICE: We check the HTTP Status before moving forward
        if (response.status === 404) {
            console.error("Test Fail: User not found (Code 404).");
            return; // We stop the function here
        }

        const data = await response.json();
        console.log("B. Data received:", data); // Does not execute if
there was an error above

    } catch (error) {
        // We "caught" the error, so the app DID NOT crash!
        console.error("Test Error: A severe network or parsing problem
occurred:", error.message);
    }
};
```

3.5 Did you know...?

💡

- **HTTP Status Codes (What a QA must know):**
  - 200 (OK): Request was successful, you received the data.
  - 201 (Created): A new resource was created on the server (e.g., you submitted a valid form).
  - 400 (Bad Request): You (the Client) sent incorrect or incomplete data.
  - 401 (Unauthorized): You are not logged in, you need to send an access token.
  - 404 (Not Found): The requested address or resource does not exist on the server.

    - 500 (Internal Server Error): The backend code crashed. You did everything right, the developers need to fix the server!
- **Playwright and Network Interception (Mocking):** In Module 7, you will learn the ultimate "magic". Instead of letting the test command ask for real data from the server, with Playwright you can intercept that "Waiter" on the way, put some fake data created by you on their tray (Mock Data), and send them back to the Client! This will help you test the Frontend application regardless of whether the Backend (Server) is functional or not at the time of the test run!

3.6 🧠 Interview Challenge / Practical Execution

🧠 Open your browser's console and execute a real request!

Your Mission:

Write an asynchronous function (Arrow Function with async) named fetchPosts.

- Inside it, use await fetch() to request the list of posts from this public API: 'https://jsonplaceholder.typicode.com/posts'.
- Extract the data using await response.json().
- Considering that the unpacked response is a list (Array), use the console.log() command to display the total number of items received (reading the .length property of the list).

(The expected answer in the console should be: "Received 100 posts.")

3.7 🤖 Engineer Mindset in the AI Era: "Don't test blind, test completely"

🌓 **QA vs. AI: Ignoring Error Codes**

- **What AI does well:** It instantly generates a Playwright API test, correctly using the equivalent command of fetch (request.get()) and correctly extracting the JSON.
- **The Trap (Where AI fails):** An unrefined AI will send the request and immediately try to check if json.name === "Alex". If the server goes down and returns a *500 Internal Server Error* (which on the network is usually an HTML error page, not a valid JSON format), the .json() parsing command will crash ugly ("Unexpected token < in JSON at position 0"), and the test report will show that the test is "Broken", instead of clearly stating that the server is down.
- **How a QA engineer thinks:** The engineer makes sure their test acts as a shield. Before doing the dangerous "unpacking" of the JSON, they force the script to validate the HTTP transaction: expect(response.status()).toBe(200);. This ensures a clear and professional test report (the test fails with an impeccable business message: "Expected 200, but got 500").
- **Your place in the market:** Writing E2E is not just about visually checking buttons moving on the screen, but checking the hidden layers (Network). Highly sought-after automation engineers are those who can validate not only that "the product appeared in the cart," but also that "the background API response that generated the product executed in less than 500ms and returned the 200 OK status." This is called **Integrated Architectural Testing**.

Chapter 4: Advanced Concepts (Bonus) - Beyond GET: Sending data (The POST Method)

Learning Objectives

🎯

- Understand the essential difference between HTTP methods GET (read) and POST (create/send).
- Learn how to configure the fetch() function with additional parameters (Method, Headers, Body).
- Transform a JavaScript Object into a network-compatible text using JSON.stringify().
- Correlate these concepts with how you will prepare test data directly via API in Playwright, bypassing the graphical interface.

4.1 When "reading" is not enough (GET vs. POST)

In the previous chapter, we sent the "Waiter" (the API) to the kitchen just to bring us the list of what already exists (e.g., the task list). This is called a **GET** request (the default one in fetch()).

But what do we do when the user fills out our form and clicks "Add Task"? The Frontend has to take that text, package it, and send it to the Server to be saved in the database. When we *send* new data to be saved, we use the HTTP **POST** method.

- **Other important methods for QA:**
  - PUT / PATCH: Used to *update* or modify an existing resource (e.g., changing a password).
  - DELETE: Used to *delete* a resource from the server.

4.2 Configuring fetch to send data

If for GET we only needed the URL, for POST we must give fetch() a second parameter: a **configuration** object (options). This object tells the server three essential things:

- **method**: What action we are taking (e.g., 'POST').
- **headers**: A note attached to the package. We must tell the server what language we are speaking. Usually, we tell it we are sending JSON data: { 'Content-Type': 'application/json' }.
- **body**: The actual data we are sending. *Attention!* On the internet, you cannot directly send a JavaScript Object. You have to transform it into a string (raw text). We use the reverse command of what we learned earlier:

JSON.stringify(myObject).

4.3 Practical Example: Creating a new Task on the Server

Here is what the complete asynchronous code looks like to send a new task to the server and print the response (usually a 201 Created status and the new task's ID).

```
const sendTaskToServer = async (newTaskName) => {
    try {
        // 1. Prepare the data (The JS Object)
        const dataPayload = {
            title: newTaskName,
            completed: false,
            userId: 1
        };

        // 2. Call fetch with the URL and the Configuration Object
        const response = await
fetch('https://jsonplaceholder.typicode.com/todos', {
            method: 'POST', // We specify that we want to create something
            headers: {
                // We tell the server to expect JSON format
                'Content-Type': 'application/json'
            },
            // Transform the JS object into Text (Stringify)
            body: JSON.stringify(dataPayload)
        });

        // 3. Check if the save was successful (Code 201 means "Created")
        if (response.status === 201) {
            const savedData = await response.json();
            console.log("Success! The server saved the task and assigned it
the ID:", savedData.id);
        } else {
            console.error("There was a problem saving. Status:",
response.status);
        }

    } catch (error) {
        console.error("Fatal network error while sending:", error.message);
```

```
    }
};

// Call the function
sendTaskToServer("Learn API testing!");
```

4.4 🤖 Engineer Mindset in the AI Era: API Setup

🌓 **QA vs. AI: Interface (UI) vs. Network (API)**

- **The Scenario:** You want to write an automated test in Playwright that verifies the **Delete a Task** function.
- **What a Junior (or basic AI) will do:** They will write the test like this:
  1. Open the add task page.
  2. Fill in the "New Task" field (UI).
  3. Click the "Save" button (UI).
  4. Search the list for the "Delete" button (UI).
  5. Click it and validate the deletion.
- **The Problem:** Steps 1-3 are just a *Setup* (preparing the conditions for the test), but they are slow (take 5 seconds) and flaky (if the add UI changes, the delete test breaks).
- **How a QA Engineer (Senior Automation) thinks:** The engineer knows the **POST** method. They will completely bypass the slow graphical interface. At the beginning of the test, in the setup hooks (e.g., beforeAll), they will launch an invisible fetch('.../todos', { method: 'POST', ... }) directly to the database, creating the task in 100 milliseconds! Then, their UI test focuses exclusively on visually validating the *Delete* button, reducing the test duration from 5 seconds to 1 second and exponentially increasing its stability.

Chapter 5: Session 5 Assignment & Extra Practice (Towards Senior Level)

5.1 Homework Assignment (Session 5 Project)

📖 **Your Task (The Business Task):**

> 📝 Until now, when opening our "Task Tracker" application, the list was always empty (or contained static data written directly in the HTML). In a real Enterprise application, when the user accesses the page, the Frontend instantly makes a request to the Backend to fetch the tasks saved in the database.

> Your task is to write a function that does exactly this: upon page load, it will download 5 tasks from a public API and draw them on the screen!

🔗 ***Execution note**: You will add this code to your existing app.js file.*

⚠️ **Technical Requirements (Acceptance Criteria for validation):**

- **Creating the asynchronous function:** Declare an asynchronous Arrow Function named loadInitialTasks = async () => { ... }.
- **Fetch API:** Inside the function, use the await fetch() command to make a GET request to the following address: 'https://jsonplaceholder.typicode.com/todos?_limit=5'. (This public API will return a list of exactly 5 test tasks).
- **Data extraction:** Unpack the received response using the .json() method (do not forget the second await!). Your variable will now be an Array of 5 Objects.
- **DOM Rendering (Combining with Session 4):**
  - Find the main list in the HTML (#active-tasks-list).
  - Use a for loop (or .forEach()) to iterate through the Array received from the server.
  - For each object in the list, dynamically create an <li> element.
  - Set the text of the <li> element to the task title coming from the network (the .title property of the JSON object).
  - Inject the <li> element into the list using appendChild().
- **Calling the function:** Simply write loadInitialTasks(); at the end of your app.js file so the function triggers automatically as soon as the script is loaded.

(***QA Analogy:** When you automate with Playwright, this exact delay caused by fetch will require your robot to use an "Auto-Waiting" assertion - e.g., await expect(page.locator('.task-item')).toHaveCount(5) - to give JavaScript time to fetch the data and render the 5 elements on the screen).*

5.2 Bonus Challenge (Optional): "Loading..." Effect and Safety (try/catch)

💣 **The Scenario:**

You are at a top company. The designer tells you: *"The request to the server might take 2 seconds. If the screen remains blank during this time, the user will think the app has frozen. Furthermore, if the server goes down, I want us to display a red error, not just crash in the console."*

🤖 **Your Mission:**

Modify the function above.

- Find the element with the ID #loading-msg (which we created in Session 4).
- On the first line of your function, make the message visible: loadingMsg.style.display = 'block';.
- Wrap the fetch and for logic inside a **try { ... }** block.
- At the end of the try block, hide the message: loadingMsg.style.display = 'none';.
- Add the **catch (error) { ... }** block. Here, if the network drops (you can test by intentionally breaking the URL), catch the error and display directly in the DOM, replacing the task list, a red text: *"Critical network error. Please try again."*

5.3 🗝️ Solution Keys (Answers & Explanations)

🚨 🛑 🚫 🚧 **Don't cheat!** Read this only after you get stuck or try to write the code yourself. This is a real interview problem for Frontend / SDET positions!

✅ **The Complete Solution (With Bonus Challenge included):**

```
// Our main asynchronous function
const loadInitialTasks = async () => {
    const taskList = document.querySelector('#active-tasks-list');
    const loadingMsg = document.querySelector('#loading-msg');

    // 1. We show the user that the system is "thinking" (UX Design)
    if (loadingMsg) loadingMsg.style.display = 'block';

    try {
        // 2. We make the network request (Waiting for the "Waiter")
        const response = await
fetch('https://jsonplaceholder.typicode.com/todos?_limit=5');

        // We check if the status is 200 OK
```

```
        if (!response.ok) throw new Error("Server Error!");

        // 3. We unpack the JSON data
        const serverTasks = await response.json();

        // 4. We iterate the list and inject it into the DOM (Exactly like
in Session 4)
        for (let i = 0; i < serverTasks.length; i++) {
            const newLi = document.createElement('li');
            newLi.className = 'task-item';
            // We fetch the 'title' property from the JSON object returned
by the API
            newLi.textContent = "☁️ " + serverTasks[i].title;

            taskList.appendChild(newLi);
        }

    } catch (error) {
        // 5. What do we do if the internet is down or the API doesn't
respond?
        console.error("Warning, the request failed:", error);
        taskList.innerHTML = '<li style="color: red; font-weight:
bold;">Critical network error. Could not load data.</li>';

    } finally {
        // The 'finally' block always executes, regardless of Try or Catch!
        // Here we always hide the "Loading..." message at the end.
        if (loadingMsg) loadingMsg.style.display = 'none';
    }
};

// We trigger the function automatically when the script runs
loadInitialTasks();
```

🎉 **Congratulations! You have completed Session 5!**

Look at the code above. This is no longer just playing around in the console; this is **Real Software Architecture**. You managed the application state (Loading), communicated asynchronously with a database (Fetch/Await), processed a payload (JSON), and fail-safely handled an exception (Try/Catch). You are ready to dominate the web environment! Playwright is next.

Live Coding Manual: Session 5 - Asynchronous JavaScript & Fetch API

> 🤖 **Guide for the Live Session:** This document contains the workspace (Sandbox) and the code snippets we will write and run *together* during the class. Each exercise corresponds to a theoretical chapter and helps us visually demonstrate the concepts of Asynchrony, Promises, and the Fetch API.

Part 1: The Playground (UI Sandbox)

📂 **Live Action:** The whole class opens VS Code, creates a live-session.html file, copies the code below, and opens it in the browser (Live Server / double-click). Here is where we will run all the experiments from Session 5!

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>S5 Live Sandbox</title>
    <style>
        body { font-family: 'Segoe UI', sans-serif; background-color:
#282c34; color: white; padding: 30px; }
        .container { max-width: 700px; margin: auto; background: #21252b;
padding: 20px; border-radius: 10px; box-shadow: 0 4px 10px rgba(0,0,0,0.5);
}
        button { background: #61afef; color: #282c34; border: none;
padding: 10px 15px; cursor: pointer; border-radius: 5px; font-weight: bold;
font-size: 14px; margin-right: 10px; }
        button.btn-danger { background: #e06c75; color: white; }
        .output-screen { background: #1e1e1e; color: #98c379; padding:
15px; margin-top: 20px; border-radius: 5px; font-family: 'Courier New',
Courier, monospace; min-height: 50px; border: 1px solid #3e4451; }
        ul { list-style: none; padding: 0; }
        li { background: #3e4451; margin-bottom: 8px; padding: 10px;
border-radius: 4px; border-left: 5px solid #c678dd; }
    </style>
</head>
<body>
    <div class="container">
        <h2>🚀 Live Module: API Testing & Asynchrony</h2>

        <div>
            <button id="btn-sync" class="btn-danger">Sync Test (Blocks
UI!)</button>
            <button id="btn-async">Async Test (SetTimeout)</button>
        </div>

        <div id="live-console" class="output-screen">
            Waiting for console commands...
        </div>

        <ul id="live-data-list">
            <!-- Data from the server will be injected here -->
        </ul>
    </div>
```

```
    <!-- We will write our live JS code here -->
    <script>
        // Global variables to manipulate the interface in the upcoming
exercises
        const screenUI = document.querySelector('#live-console');
        const htmlList = document.querySelector('#live-data-list');
    </script>
</body>
</html>
```

Part 2: Live Exercises

As we go through the theory, we will add these functions directly into the <script> tag of the live-session.html file and call them to demonstrate the behavior live.

🟤 For Chapter 1: The Blocking Demonstration (Sync vs Async)

🚀 **Live Scenario:** We will see how a heavy "while loop" (synchronous) freezes the browser, whereas setTimeout (asynchronous) leaves the interface responsive.

🤔 ❓ **How to actually test it:**

- In the live-session.html file, right below the const htmlList = ... line, add the code below.
- Save the file and refresh the page in the browser.
- Click the red **Sync Test** button and try to *immediately* select the text on the page with your mouse. You will notice everything has frozen; you cannot select absolutely anything for 3 seconds! (If you are on Chrome, the yellow text won't even render until the blockage is over).
- Then, click the blue **Async Test** button and try selecting text again. The interface is perfectly fluid, even though the script is counting 3 seconds in the background to deliver the "pizza".

Code to add inside the <script> tag:

```
const btnSync = document.querySelector('#btn-sync');
const btnAsync = document.querySelector('#btn-async');

// 1. Synchronous (Blocking) Approach - "Processing Hell"
btnSync.addEventListener('click', () => {
    screenUI.innerHTML = "<span style='color:#e06c75;'>⏳ I froze the page
for 3 seconds! Try selecting this text...</span>";
```

```
    // Forcing a real blockage of the Main Thread
    const start = Date.now();
    while(Date.now() - start < 3000) { /* CPU at 100%, total freeze! Event
Loop is blocked! */ }

    screenUI.innerHTML = "✅ The page is unfrozen.";
});

// 2. Asynchronous (Non-blocking) Approach - "Delegation"
btnAsync.addEventListener('click', () => {
    screenUI.innerHTML = "⏳ Request sent in the background (Async). You
can select text, you are not blocked!";

    // Delegate the waiting to the Web API
    setTimeout(() => {
        screenUI.innerHTML = "🍕 Pizza arrived (after 3 background
seconds)!";
    }, 3000);
});
```

🔘 For Chapter 2: Promises & async/await

🚀 **Live Scenario:** We will create our own Promise (a Tracking Number / AWB) that resolves after 2 seconds and consume it using the Playwright syntax (await).

🤔 ❓ **How to actually test it:**

- Continue writing the code below inside the <script> tag of your live-session.html file.
- Save the file and refresh the page.
- Open the **Console (F12)** in your browser.
- To trigger the function, type the command executePromiseTest() in the console and press Enter.
- Look at the black UI screen on the page: you will instantly see the message "Waiting for package (State: Pending)...".
- Count 2 seconds and watch how the text automatically changes to "💻 Gaming Laptop delivered...", visually demonstrating how await "paused" only that specific function until the Promise was resolved.

Code to add inside the <script> tag:

```
// 1. Create the Promise (Emulate a network call taking 2 seconds)
const orderLaptop = () => {
    return new Promise((resolve, reject) => {
        setTimeout(() => {
            resolve("💻 Gaming Laptop successfully delivered!");
        }, 2000);
    });
};

// 2. Modern test function (Exactly how we will write it in Playwright)
const executePromiseTest = async () => {
    screenUI.innerHTML = "Waiting for package (State: Pending)...";

    // The 'await' keyword PAUSES strictly this function
    const trackingResult = await orderLaptop();

    // Execution continues only after State: Fulfilled
    screenUI.innerHTML = trackingResult;
};
```

🟤 For Chapter 3: The First Network Request (Fetch GET)

🚀 **Live Scenario:** Fetch REAL data from the internet. We will query the public JSONPlaceholder API to get details of a single User and inject them into our screenUI div.

🤔 ❓ How to actually test it:

- Copy the function below into live-session.html, save, and refresh.
- Go to the F12 Console, type the command checkUserProfile() and press Enter.
- Look at the UI screen. You will see "Contacting server..." for a fraction of a second.
- The message will be replaced almost instantly by the *real* data of Leanne Graham (name, email, website) that was just downloaded live from the JSONPlaceholder internet database.

Code to add inside the <script> tag:

```
const checkUserProfile = async () => {
    screenUI.innerHTML = "Contacting server (Fetch)...";

    // 1. Wait for the "Waiter" to bring the HTTP Package from the kitchen
    const httpResponse = await
fetch('https://jsonplaceholder.typicode.com/users/1');
```

```
    // 2. Wait for unpacking the text into a JavaScript Object (JSON)
    const userData = await httpResponse.json();

    // 3. Validation & Rendering on our HTML screen
    screenUI.innerHTML = `
        ✓ Data received from API:<br><br>
        <strong>Name:</strong> ${userData.name} <br>
        <strong>Email:</strong> ${userData.email} <br>
        <strong>Website:</strong> ${userData.website}
    `;
};
```

🔘 For Chapter 4 (Bonus): Sending Data to the Server (Fetch POST)

📌**Live Scenario:** Simulate an E2E "Account Creation" command via API, bypassing the graphical interface. Set the Method, Headers, and Body, and display the server's status on our interface.

🤔 ❓ **How to actually test it:**

- Add the function in live-session.html, save, and refresh.
- In the F12 Console, run createTaskViaAPI().
- The sending message will appear on the black screen, quickly followed by the green success message.
- Look closely at the ID printed on the screen: the server will return ID: 101, practically proving it received, processed, and successfully simulated adding your object (payload).

Code to add inside the <script> tag:

```
const createTaskViaAPI = async () => {
    screenUI.innerHTML = "Sending new data to the database...";

    // 1. Our JavaScript Object
    const payload = {
        title: "Learn Playwright API",
        body: "Direct backend testing",
        userId: 99
    };
```

```
    // 2. POST request (Create)
    const response = await
fetch('https://jsonplaceholder.typicode.com/posts', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify(payload) // Translate JS Object to Internet
Text
    });

    // 3. Extract the response saved by the server
    if (response.status === 201) { // 201 = Created
        const savedData = await response.json();
        screenUI.innerHTML = `✓ Success! Server replied with Status 201.
<br> New task received ID: <strong
style="color:white;">${savedData.id}</strong>.`;
    }
};
```

🔘 For Chapter 5: End-to-End & Controlled Failure (try/catch)

🧨 **Live Scenario:** Fetch a list of 3 items and inject them into the htmlList (<ul>). Force a network error (wrong URL) to see how try/catch protects the app from crashing.

🤔 ❓ **How to actually test it:**

- Add the final code, save, and refresh.
- Run in the F12 Console: testListLoading().
- Notice how the script runs the WRONG_URL variant. The interface *does not freeze*, but immediately displays the red text cleanly caught by the catch block ("❌ CONTROLLED ERROR...").
- Now, edit your code in VS Code: modify the fetch line to use the valid parameter: const response = await fetch(CORRECT_URL);.
- Save, refresh, and run testListLoading() again in the console. The try block executes successfully this time, rendering the 3 <li> elements as a list in the UI!

Code to add inside the <script> tag:

```
const testListLoading = async () => {
```

```
    htmlList.innerHTML = '';
    screenUI.innerHTML = "Loading list...";

    try {
        // INTENTIONALLY BREAK THE URL for the wrong variant
        const CORRECT_URL =
'https://jsonplaceholder.typicode.com/todos?_limit=3';
        const WRONG_URL =
'https://jsonplaceholder.typicode.com/todosXXX?_limit=3';

        // <--- Test one by one with both URLs by changing the variable
below
        const response = await fetch(WRONG_URL);

        if (!response.ok) {
            throw new Error(`Server replied with fatal error:
${response.status}`);
        }

        const taskList = await response.json();

        // DOM Rendering using the htmlList variable declared at the
beginning
        taskList.forEach(task => {
            const li = document.createElement('li');
            li.textContent = `[ID: ${task.id}] ${task.title}`;
            htmlList.appendChild(li);
        });

        screenUI.innerHTML = "<span style='color:#98c379;'>All data loaded
and drawn on screen!</span>";

    } catch (error) {
        // We end up here if the internet drops or the URL is wrong
        console.error("Attention QA, error caught:", error);

        screenUI.innerHTML = `<span style='color:#e06c75;'>❌ CONTROLLED
ERROR: ${error.message}</span>`;
        htmlList.innerHTML = "<li style='border-color: #e06c75; color:
#e06c75;'>System unavailable. List could not be loaded.</li>";
    }
};
```

Advanced Workbook: Session 5 - Asynchronous JavaScript (Extra Practice)

> 📈 This material is designed for individual deep-diving into asynchrony concepts, going beyond the basic exercises from the live session. We will work on a different scenario: a "Server Monitoring Dashboard".
>
> Here you will find **8 practical exercises**, including advanced concepts (like parallelization with Promise.all()), which you will execute directly in the console to manipulate the application's interface.

Part 1: Application Setup (The Sandbox)

Step 1: Save the code below in a file named dashboard.html using VS Code and open it in Chrome. **Step 2:** All 8 exercises will be run by pasting the code into the **Console (F12)**, observing how the page UI reacts.

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Server Monitoring Dashboard</title>
    <style>
        body { font-family: 'Segoe UI', Tahoma, sans-serif; background-
color: #1e1e2f; color: #fff; padding: 20px; }
        .dashboard { background: #2a2a40; padding: 25px; border-radius:
10px; box-shadow: 0 10px 20px rgba(0,0,0,0.3); max-width: 800px; margin:
auto; }
        .panel { background: #1e1e2f; border: 1px solid #444; border-
radius: 6px; padding: 15px; margin-bottom: 15px; }
        button { background: #00d2d3; color: #1e1e2f; border: none;
padding: 10px 20px; cursor: pointer; border-radius: 4px; font-weight: bold;
font-size: 14px; margin-right: 10px; transition: 0.2s;}
        button:hover { background: #01a3a4; }
        button.btn-danger { background: #ff6b6b; color: white; }
        #log-screen { font-family: 'Courier New', monospace; background:
#000; color: #10ac84; padding: 15px; min-height: 100px; border-radius: 4px;
margin-top: 15px; overflow-y: auto; max-height: 200px;}
        .server-grid { display: grid; grid-template-columns: 1fr 1fr; gap:
10px; }
        .server-card { padding: 10px; background: #3d3d5c; border-left: 5px
solid #5f27cd; border-radius: 4px; }
    </style>
</head>
<body>
    <div class="dashboard">
        <h2>📊 Server Monitor (QA Practice)</h2>
        <p>Press <strong>F12</strong>, open the <strong>Console</strong>
tab and run the 8 exercises one by one.</p>

        <div class="panel">
            <button id="btn-stress" class="btn-danger">Stress Test
(Sync)</button>
            <input type="text" id="test-input" placeholder="Try typing
here..." style="padding: 10px; width: 200px;">
        </div>

        <div class="panel">
            <h3>Terminal Logs</h3>
```

```
            <div id="log-screen">System ready. Waiting for async
commands...</div>
        </div>

        <div class="panel">
            <h3>Server Data (Live API)</h3>
            <div id="server-data" class="server-grid">
                <!-- Data fetched via API will be injected here -->
            </div>
        </div>
    </div>

    <script>
        // Global helper variables for your exercises
        const terminal = document.querySelector('#log-screen');
        const serverGrid = document.querySelector('#server-data');

        // Utility function to easily write in the black terminal
        const writeToTerminal = (message, color = '#10ac84') => {
            terminal.innerHTML += `<br><span style="color: ${color};">>>>
${message}</span>`;
            terminal.scrollTop = terminal.scrollHeight; // Auto-scroll down
        };
    </script>
</body>
</html>
```

Part 2: The Exercise Suite (Run in F12 Console)

💻 Exercise 1: The Sync Nightmare (Event Loop Blocked)

🚀 **Scenario:** We demonstrate how an intense mathematical action completely blocks the page.

🤔 ❓ **How to actually test it:**

- Copy the code below and run it in the F12 console (press Enter).
- Immediately, go to the page and physically click the red "Stress Test (Sync)" button.
- Try rapidly clicking in the text field next to the button and typing something.
- You will notice that the browser has completely "frozen" for 4 seconds! You cannot type

anything until it unfreezes.

```
document.querySelector('#btn-stress').addEventListener('click', () => {
    writeToTerminal("Starting stress test (PAGE IS BLOCKED!)...",
"#ff6b6b");
    const start = Date.now();
    // Block the processor for 4 seconds
    while (Date.now() - start < 4000) {}
    writeToTerminal("Test complete. You can type in the input again.");
});
```

💻 Exercise 2: Asynchronous Chaining (setTimeout)

🚀 **Scenario:** We simulate the steps of an E2E test (Setup -> Test -> Teardown) running in the background without blocking the UI.

🤔 ❓ **How to actually test it:**

- Copy the code and run it in the console.
- Look at the black "Terminal Logs" screen on the page.
- While the messages appear one by one (spaced by one second), click in the white input and type text. It works perfectly, the interface is not blocked at all!

```
const runBackgroundTest = () => {
    writeToTerminal("1. [Async] Initializing browser...");

    setTimeout(() => {
        writeToTerminal("2. [Async] Navigated to page (after 1s).",
"#feca57");

        setTimeout(() => {
            writeToTerminal("3. [Async] Element found and clicked (after
2s).", "#feca57");

            setTimeout(() => {
                writeToTerminal("4. [Async] Assertion passed! Test
completed successfully (after 3s).");
            }, 1000);
```

```
        }, 1000);

    }, 1000);
};

runBackgroundTest();
```

💻 Exercise 3: Creating a Promise from Scratch

🚀 **Scenario:** We build our own "Promise" mechanism that simulates checking a server's health status.

🤔 ❓ **How to actually test it:**

- Copy and run the code in the console.
- This time, look at the response printed by the F12 Console itself (not the terminal on the page).
- You will see a Promise { <pending> } object. Expand it (click the small arrow next to it) after 2 seconds have passed. You will see its internal state ([[PromiseState]]) has changed to fulfilled.

```
const pingServer = () => {
    return new Promise((resolve, reject) => {
        setTimeout(() => {
            // Simulate the server successfully responding after 2 seconds
            resolve({ status: 200, message: "DB Server is online and
running." });
        }, 2000);
    });
};

const myPromise = pingServer();
console.log("Promise state at this moment:", myPromise);
```

💻 Exercise 4: Consuming the Promise (async/await)

🚀 **Scenario:** Now we use the Playwright syntax (await) to actually wait for the result of the Promise created in Exercise 3 and display it in our terminal.

🤔 ❓ **How to actually test it:**

- *Note: Ensure you have already run Exercise 3 (so the pingServer function exists in memory).*
- Run the code below in the console.
- Look at the black terminal on the page. It will say "Sending ping...", wait exactly 2 seconds on pause, then print the response extracted from the Promise.

```
const checkHealthStatus = async () => {
    writeToTerminal("Sending ping to the database...");

    // The code waits here for the 2 seconds
    const response = await pingServer();

    writeToTerminal(`Response received: Code ${response.status} -
${response.message}`);
};

checkHealthStatus();
```

💻 Exercise 5: Fetch GET - Bringing Real Data

🚀 **Scenario:** We use the Fetch API to bring the details of a QA engineer from a real test server (JSONPlaceholder) and display it as an HTML card.

🤔 ❓ **How to actually test it:**

- Run the code in the console.
- Look at the "Server Data (Live API)" section in the bottom-right of the page.
- A new card with "Engineer: Chelsey Dietrich" will appear instantly on the screen, representing real data extracted from the internet right now!

```
const fetchEngineerDetails = async () => {
```

```
    writeToTerminal("Querying API for user 5...");

    const response = await
fetch('https://jsonplaceholder.typicode.com/users/5');
    const user = await response.json();

    // Draw the real data extracted from JSON into the DOM
    serverGrid.innerHTML += `
        <div class="server-card">
            <h4 style="margin: 0; color: #00d2d3;">Engineer:
${user.name}</h4>
            <p style="margin: 5px 0; font-size: 12px;">Email:
${user.email}</p>
            <p style="margin: 5px 0; font-size: 12px;">Company:
${user.company.name}</p>
        </div>
    `;
    writeToTerminal("Data successfully rendered on screen.");
};

fetchEngineerDetails();
```

💻 Exercise 6: Fetch POST - Reporting a Bug over the Network

🚀 **Scenario:** We simulate adding a Bug Report directly to the database via API, configuring the method, headers, and transforming the JS object into a string (JSON.stringify).

🤔 ❓ **How to actually test it:**

- Run the code in the console.
- Look at the black log terminal.
- You will see the pink success message confirming that the server processed your object and allocated the new ID to it: 101.

```
const reportBugAPI = async () => {
    writeToTerminal("Sending Bug Report to server (POST)...");

    const bugPayload = {
```

```
        title: "Login button not working on small resolutions",
        body: "Severity: High. Reproducibility: 100%.",
        userId: 99
    };

    const response = await
fetch('https://jsonplaceholder.typicode.com/posts', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify(bugPayload)
    });

    if (response.status === 201) {
        const confirmation = await response.json();
        writeToTerminal(`Success! Bug saved with ID: ${confirmation.id}`,
'#f368e0');
    }
};

reportBugAPI();
```

💻 Exercise 7: Resilience with Try/Catch

🚀 **Scenario:** What happens if the API goes down? We intentionally break the URL. If we didn't have try/catch, the app would crash silently. We intercept the error and display a red report.

🤔 ❓ **How to actually test it:**

- Run the code in the console.
- Look at the black terminal on the page.
- Because we intentionally broke the link, the code will cleanly "jump" into the catch block and display the red error message. Your page did not freeze, and everything is handled professionally.

```
const safeNetworkTest = async () => {
    try {
        writeToTerminal("Initializing stable test...");
```

```
        // INTENTIONALLY WRONG URL (typo in 'users')
        const response = await
fetch('https://jsonplaceholder.typicode.com/usersXXX/1');

        if (!response.ok) {
            // We forcefully throw an error that the catch block below will
intercept
            throw new Error(`Server returned HTTP error:
${response.status}`);
        }

        const data = await response.json();
        writeToTerminal("Data fetched: " + data.name);

    } catch (error) {
        writeToTerminal(`❌ CONTROLLED TEST FAILURE: ${error.message}`,
'#ff6b6b');
    }
};

safeNetworkTest();
```

💻 Exercise 8: Advanced Concept - Parallelization (Promise.all)

🚀 **Senior level scenario:** You have to verify 3 elements from 3 different addresses. If you wait for them one by one with await, it takes a long time. With Promise.all, you send all 3 over the network at the same time!

🤔 **? How to actually test it:**

- Run the code in the console.
- Look closely at the "Server Data" section and the black terminal.
- Notice that, although we requested data for 3 totally different users, all 3 cards appear on the screen EXACTLY at the same time, demonstrating super-fast parallel execution!

```
const downloadMassData = async () => {
    writeToTerminal("Launching 3 network requests SIMULTANEOUSLY
```

```
(Promise.all)...", "#feca57");

    try {
        // We prepare the 3 Fetch Promises (we don't put await in front of
them yet!)
        const request1 =
fetch('https://jsonplaceholder.typicode.com/users/1').then(r => r.json());
        const request2 =
fetch('https://jsonplaceholder.typicode.com/users/2').then(r => r.json());
        const request3 =
fetch('https://jsonplaceholder.typicode.com/users/3').then(r => r.json());

        // Now we wait for ALL 3 to return from the internet
        const results = await Promise.all([request1, request2, request3]);

        // 'results' is an Array with the 3 JSON objects! We can draw them:
        results.forEach(user => {
            serverGrid.innerHTML += `
                <div class="server-card" style="border-color: #feca57;">
                    <h4 style="margin: 0;">Multi-Fetch: ${user.name}</h4>
                    <p style="margin: 5px 0; font-size: 12px;">City:
${user.address.city}</p>
                </div>
            `;
        });

        writeToTerminal("Parallelization successful! 3 users loaded
instantly.");

    } catch (error) {
        writeToTerminal(`Parallelization error: ${error.message}`,
'#ff6b6b');
    }
};

downloadMassData();
```